



ASSIGNMENT 7.1

De curl a un cliente Python

Consumir una API de predicción como caja negra

DURACION	MODALIDAD	PREREQUISITOS
75-90 min	Parejas, assignment posterior a la clase 1	Microejercicios HTTP/JSON y API local preparada

1. Encargo

Observar el contrato de una API de predicción y completar un cliente Python que trate de forma explícita petición, status, transporte y JSON.

Regla de continuidad

La API de bombas se entrega como caja negra: se ejecuta, pero no se modifica.

2. Punto de partida

- El recorrido comienza con curl y termina con cinco pruebas del cliente en verde.
- Postman es una ampliación opcional y no forma parte de la calificación.

3. Material que recibes

- exercises/02-api-client-lab/problem/starter/client.py con los marcadores 3-6
- Cinco pruebas con sesiones y respuestas controladas
- API local y muestras JSON válidas e inválidas
- Guía curl con variantes Bash y PowerShell

4. Tareas

01 Prepara dos terminales - Desde la raíz del módulo sincroniza el entorno, arranca la API en la primera terminal y conserva la segunda para observación y tests.

02 Observa el contrato - Comprueba GET /health, un POST válido y un POST inválido; registra 200, 200 y 422, request_id y campo rechazado.

03 Ejecuta el estado inicial - Lanza la suite del starter y confirma cinco pruebas rojas por el NotImplementedError intencional.

04 Implementa la petición - Usa POST /v1/predictions, json=payload, Accept application/json y timeout explícito sin cambiar la firma pública.

05 Traduce fallos y valida - Aplica raise_for_status; traduce HTTP, timeout y ConnectionError a PredictionClientError; decodifica JSON y exige un objeto.

06 Verifica y reúne evidencia - Obtén cinco pruebas verdes y conserva las tres respuestas curl y una explicación sobre el reintento de POST.

5. Entrega

Evidencia	Que debe permitir comprobar
Cliente	starter/client.py completado sin modificar la API de caja negra.
Tests	Salida final con cinco pruebas verdes.
Evidencia HTTP	Salud, POST 200 y POST 422 con status, request_id y campo rechazado.
Decisión	Una frase que justifique por qué no se reintenta un POST ciegamente.

6. Criterios de aceptacion

- [] La petición usa la ruta, cuerpo JSON, cabecera Accept y timeout indicados.
- [] raise_for_status impide interpretar un 422 como una predicción correcta.
- [] HTTPError conserva el status; timeout y ConnectionError se distinguen.
- [] Una respuesta 2xx solo se acepta si contiene un objeto JSON.
- [] Las cinco pruebas pasan y la evidencia contiene 200, 200 y 422.
- [] No se modifica el servidor ni se copia la solución docente.

7. Rubrica orientativa

Criterio	Puntos
Petición HTTP	3
Tratamiento de errores	2
Validación de respuesta	2
Tests y evidencias	2
Explicación y reproducibilidad	1

8. Comprobacion

```
# Terminal 1 - servidor
cd semana7/modules/07-http-rest-clients
uv sync
uv run python examples/pump-maintenance-api/server.py

# Terminal 2 - Bash o Git Bash
cd semana7/modules/07-http-rest-clients
BASE_URL=http://127.0.0.1:8000
curl -i "$BASE_URL/health"
curl -i -X POST "$BASE_URL/v1/predictions" \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  --data-binary @examples/pump-maintenance-api/samples/prediction-valid.json
# Repite con prediction-invalid.json

# Terminal 2 - tests
uv run python -m unittest discover -s exercises/02-api-client-lab/problem/starter -p "test_*.py" -v
```

9. Notas y limites

- No abras ni modifiques el servidor; en S7 se observa como caja negra.
- No implementes FastAPI ni autenticación: el backend comienza en S8.
- No uses Wine, Streamlit ni HttpInferenceGateway; pertenecen a la Clase 2.
- PowerShell: usa curl.exe y los comandos equivalentes de examples/curl/README.md.

Referencia en el repositorio

semana7/modules/07-http-rest-clients/exercises/02-api-client-lab/problem/README.md
 semana7/modules/07-http-rest-clients/guides/class-1-practices.md
 semana7/modules/07-http-rest-clients/examples/curl/README.md